

```

// -----
// Ex1 - index.html *Code by myself*
// -----
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Audio Processing App</title>
  <!-- Include p5.js library -->
  <script src="libraries/p5.js"></script>
  <script src="libraries/p5.sound.js"></script>
  <script src="libraries/p5.sound.min.js"></script>
  <style>
    body {
      margin: 0;
      font-family: Arial, sans-serif;
      text-align: center;
      background: linear-gradient(135deg, #7d9eff, #95e0d9);
      color: #333;
    }

    h1 {
      margin: 20px 0;
      font-size: 24px;
      color: #fff;
      text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.5);
    }

    button {
      font-size: 14px;
      padding: 10px 20px;
      margin: 5px;
      border: none;
      border-radius: 8px;
      background-color: #789fd4;
      color: white;
      cursor: pointer;
      transition: background-color 0.3s;
    }

    button:hover {
      background-color: #d7dce0;
    }

    button:active {
      background-color: #367c39;
    }

    input[type="range"] {
      width: 100%;
    }
  </style>

```

```

        .filters-wrapper {
            display: flex; /* Enable flexbox for side-by-side layout
*/
            gap: 20px; /* Space between the containers */
            justify-content: center; /* Center the containers
horizontally */
            margin: 20px 0; /* Add some margin above and below */
        }

        .filter-container {
            border: 5px solid #6781a5;
            border-radius: 8px;
            padding: 15px;
            margin: 20px auto;
            background-color: #f9f9f9;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            max-width: 400px;
        }

        .title-and-audio {
            display: flex; /* Use flexbox for horizontal alignment
*/
            flex-direction: column; /* Stack title and audio
controls vertically */
            align-items: center; /* Center items horizontally */
        }

        .filter-container h3 {
            margin-top: 0;
            text-align: center;
            font-size: 18px;
            color: #333;
        }

        .filter-container div {
            margin-bottom: 15px;
        }

        .filter-container label {
            font-size: 14px;
            color: #555;
        }

        .filter-container span {
            font-size: 12px;
            color: #777;
            margin-left: 10px;
        }

        .filter-container button {
            margin-top: 10px;
        }

        #audioSourceControls {

```

```

        text-align: center;
        margin: 10px 0;
        background: none;
        border: none;
        padding: 0;
    }

    #volumeValue {
        font-size: 14px;
        color: #555;
        margin-left: 10px;
    }
</style>
</head>
<body>
    <div class="title-and-audio">
        <h1>Audio Processing App</h1>
        <div id="audioSourceControls" class="audio-source-
container">
            <label for="audioSourceSelector">Select Audio Source:</
label>
            <select id="audioSourceSelector">
                <option value="file">Pre-recorded File</option>
                <option value="mic">Microphone</option>
            </select>
        </div>
    </div>
    <div class="container">
        <!-- Buttons for the application -->
        <button id="playButton">Play</button>
        <button id="pauseButton">Pause</button>
        <button id="stopButton">Stop</button>
        <button id="skipStartButton">Skip to Start</button>
        <button id="skipEndButton">Skip to End</button>
        <button id="loopButton">Loop: Off</button>
        <button id="muteButton">Mute: Off</button>
        <button id="recordButton">Record</button>
        <input type="range" id="volumeSlider" min="0" max="2"
step="0.01" value="1">
        <span id="volumeValue">Master Volume Level = 0.5x</span>
    </div>
    <div class="filters-wrapper">
        <div id="filterContainer" class="filter-container">
            <h3>Filter Controls</h3>
            <button id="filterToggleButton">Filter: Off</button>
            <label for="filterTypeSelector">Filter Type:</label>
            <select id="filterTypeSelector">
                <option value="lowpass">Low-Pass</option>
                <option value="highpass">High-Pass</option>
                <option value="bandpass">Band-Pass</option>
            </select>
        </div>
        <label for="freqSlider">Frequency (Hz):</label>
        <input type="range" id="freqSlider" min="10"

```

```

max="22050" step="1">
    <span id="freqValue">1000</span>
</div>
<div>
    <label for="resSlider">Resonance (Q):</label>
    <input type="range" id="resSlider" min="0" max="10"
step="0.1">
        <span id="resValue">1</span>
</div>
<div>
    <label for="dryWetSlider">Dry/Wet Mix:</label>
    <input type="range" id="dryWetSlider" min="0"
max="1" step="0.01" value="0.5">
        <span id="dryWetValue">0.5</span>
        <small>(0 = Dry, 1 = Wet)</small>
</div>
<div>
    <label for="outputLevelSlider">Output Level:</label>
    <input type="range" id="outputLevelSlider" min="0"
max="2" step="0.01" value="1">
        <span id="outputLevelValue">1</span>
        <small>(0 = Mute, 1 = Original, >1 = Amplified)</
small>
</div>
</div>

<div id="waveshaperContainer" class="filter-container">
    <h3>Waveshaper Distortion Controls</h3>
    <button id="waveshaperToggleButton">Waveshaper: Off</
button>
    <div>
        <label for="distortionAmountSlider">Distortion
Amount:</label>
        <input type="range" id="distortionAmountSlider"
min="0" max="1" step="0.01" value="0.5">
        <span id="distortionAmountValue">0.5</span>
    </div>
    <div>
        <label
for="waveshaperOversampleButton">Oversample:</label>
        <button id="waveshaperOversampleButton">Oversample:
Off</button>
    </div>
    <div>
        <label for="waveshaperDryWetSlider">Dry/Wet Mix:</
label>
        <input type="range" id="waveshaperDryWetSlider"
min="0" max="1" step="0.01" value="0.5">
        <span id="waveshaperDryWetValue">0.5</span>
    </div>
    <div>
        <label for="waveshaperOutputSlider">Output Level:</
label>
        <input type="range" id="waveshaperOutputSlider"

```

```

min="0" max="2" step="0.01" value="1">
    <span id="waveshaperOutputValue">1</span>
</div>
</div>
<div id="delayContainer" class="filter-container">
    <h3>Delay Effect Controls</h3>
    <button id="delayToggleButton">Delay: Off</button>
    <div>
        <label for="delayTimeSlider">Delay Time (s):</label>
        <input type="range" id="delayTimeSlider" min="0.1"
max="5" step="0.1" value="0.5">
        <span id="delayTimeValue">0.5s</span>
    </div>
    <div>
        <label for="feedbackSlider">Feedback (%):</label>
        <input type="range" id="feedbackSlider" min="0"
max="1" step="0.01" value="0.5">
        <span id="feedbackValue">50%</span>
    </div>
    <div>
        <label for="filterSlider">Filter Frequency (Hz):</
label>
        <input type="range" id="filterSlider" min="200"
max="5000" step="50" value="2300">
        <span id="filterValue">2300Hz</span>
    </div>
</div>
<div id="compressorContainer" class="filter-container">
    <h3>Dynamic Compressor Controls</h3>
    <button id="compressorToggleButton">Compressor: Off</
button>
    <div>
        <label for="thresholdSlider">Threshold (dB):</label>
        <input type="range" id="thresholdSlider" min="-60"
max="0" step="1" value="-24">
        <span id="thresholdValue">-24</span>
    </div>
    <div>
        <label for="ratioSlider">Ratio:</label>
        <input type="range" id="ratioSlider" min="1"
max="20" step="1" value="4">
        <span id="ratioValue">4</span>
    </div>
    <div>
        <label for="attackSlider">Attack (ms):</label>
        <input type="range" id="attackSlider" min="0"
max="1000" step="10" value="20">
        <span id="attackValue">20</span>
    </div>
    <div>
        <label for="releaseSlider">Release (ms):</label>
        <input type="range" id="releaseSlider" min="0"
max="1000" step="10" value="200">
        <span id="releaseValue">200</span>
    </div>

```

```

        </div>
        <div>
            <label for="kneeSlider">Knee (dB):</label>
            <input type="range" id="kneeSlider" min="0" max="40"
step="1" value="10">
            <span id="kneeValue">10</span>
        </div>
        <div>
            <label for="compressorDryWetSlider">Dry/Wet Mix:</
label>
            <input type="range" id="compressorDryWetSlider"
min="0" max="1" step="0.01" value="0.5">
            <span id="compressorDryWetValue">0.5</span>
            <small>(0 = Dry, 1 = Wet)</small>
        </div>
        <div>
            <label for="compressorOutputLevelSlider">Output
Level:</label>
            <input type="range" id="compressorOutputLevelSlider"
min="0" max="2" step="0.01" value="1">
            <span id="compressorOutputLevelValue">1</span>
        </div>
    </div>
    <div id="reverbContainer" class="filter-container">
        <h3>Reverb Controls</h3>
        <button id="reverbToggleButton">Reverb: Off</button>
        <div>
            <label for="reverbDurationSlider">Reverb Duration
(s):</label>
            <input type="range" id="reverbDurationSlider"
min="0.1" max="10" step="0.1" value="2">
            <span id="reverbDurationValue">2</span>
        </div>
        <div>
            <label for="reverbDecaySlider">Decay Rate:</label>
            <input type="range" id="reverbDecaySlider" min="0"
max="1" step="0.01" value="0.5">
            <span id="reverbDecayValue">0.5</span>
        </div>
        <div>
            <label for="reverbDryWetSlider">Dry/Wet Mix:</label>
            <input type="range" id="reverbDryWetSlider" min="0"
max="1" step="0.01" value="0.5">
            <span id="reverbDryWetValue">0.5</span>
        </div>
        <div>
            <label for="reverbOutputSlider">Output Level:</
label>
            <input type="range" id="reverbOutputSlider" min="0"
max="2" step="0.01" value="1">
            <span id="reverbOutputValue">1</span>
        </div>
        <button id="reverbReverseButton">Reverse: Off</button>
    </div>

```

```

    </div>
    <script src="sketch.js"></script>
</body>
</html>
// -----
// Ex1 - sketch.js *Code by myself*
// -----
let sound; // Variable to store the audio file
let mic; // microphone input
let fft;
let processedFFT;
let currentSource = 'file'; // default source is pre-recorded file

let filter; // Single filter object
let filterEnabled = false; // State of the filter toggle

let waveshaper; // Waveshaper
let waveshaperEnabled = false; // state of waveshaper toggle
let oversampleEnabled = false; // State of the Oversample toggle

let delay; // delay effect
let delayEnabled = false; // state of delay toggle

let compressor; // Dynamic compressor
let compressorEnabled = false; // State of the compressor toggle

let reverb; // reverb effect
let reverbEnabled = false; // State of the reverb toggle
let reverbReverseEnabled = false; // State of the reverse toggle

let loopEnabled = false; // State of the Loop button
let muted = false; // State of the Mute button
let recorder, recordingFile; // For recording functionality
let isRecording = false; // State of the recording button

function preload() {
    // Load the pre-recorded sound file from the sounds folder
    sound = loadSound('sounds/guitar.wav');
}

function setup() {
    //initialise microphone input
    mic = new p5.AudioIn();

    // Initialise audio effects
    filter = new p5.Filter();
    waveshaper = new p5.Distortion(0.5, 'none'); // Default amount:
0.5, no oversampling
    delay = new p5.Delay();
    compressor = new p5.Compressor();
    reverb = new p5.Reverb();

    // Set initial delay parameters
    delay.process(sound, 0.5, 0.5, 2300); // source, delayTime,

```

```

feedback, lowPassFrequency
  delay.amp(1); // Set the output volume of the delay effect

  // Connect the filter to the sound
  sound.disconnect(); // Disconnect sound from Master output
  sound.connect(); // Route sound directly to master output

  // Initialize SoundRecorder and SoundFile
  recorder = new p5.SoundRecorder();
  recordingFile = new p5.SoundFile();

  // Connect the recorder to the master output
  recorder.setInput();

  // Bind Audio source controls
  bindAudioSourceControls();

  // Bind HTML buttons to playback controls and audio effects
  bindPlaybackControls();
  //bindLowPassFilterControls();
  bindWaveshaperControls();
  bindDelayControls();
  bindCompressorControls();
  bindReverbControls();
  bindFilterControls(); // Bind the filter type selector

  // Initialize FFT object for frequency analysis
  fft = new p5.FFT();
  fft.setInput(sound); // Set the input to the sound file
  processedFFT = new p5.FFT();
  processedFFT.setInput(); // Default to no processed input
initially

  // Create a canvas for the visualizer
  createCanvas(windowWidth, 200); // Adjust the height as needed
  background(0); // Set background to black
}

function draw() {
  // Clear the canvas with a black background
  background(0);

  // Draw original audio spectrum
  let spectrum = fft.analyze(); // Analyze the frequency spectrum

  noStroke();
  fill(0, 255, 0); // Set fill color to green

  // Draw the frequency bars for original audio
  for (let i = 0; i < spectrum.length; i++) {
    let x = map(i, 0, spectrum.length, 0, width / 2); // Map
frequency index to left half of canvas
    let h = -height + map(spectrum[i], 0, 255, height, 0); //
Map amplitude to height

```



```

        rect(x, height, width / (2 * spectrum.length), h); // Draw
rectangle for each frequency band
    }

    // Draw processed audio spectrum
    let processedSpectrum = processedFFT.analyze(); // Analyze the
processed frequency spectrum
    fill(255, 0, 0); // Set fill color to red for processed audio

    // Draw the frequency bars for processed audio
    for (let i = 0; i < processedSpectrum.length; i++) {
        let x = map(i, 0, processedSpectrum.length, width / 2,
width); // Map frequency index to right half of canvas
        let h = -height + map(processedSpectrum[i], 0, 255, height,
0); // Map amplitude to height
        rect(x, height, width / (2 * processedSpectrum.length),
h); // Draw rectangle for each frequency band
    }
}

// Audio source controls
function bindAudioSourceControls() {
    const audioSourceSelector =
document.getElementById('audioSourceSelector');

    // Handle audio source change
    audioSourceSelector.addEventListener('change', (e) => {
        const selectedSource = e.target.value;

        if (selectedSource === 'mic') {
            // Switch to microphone input
            if (!mic.enabled) {
                mic.start();
                mic.amp(0.1); // Set the gain to a lower level
            }
            sound.stop(); // Stop pre-recorded file if playing
            mic.connect(); // Connect microphone input to master
output
            currentSource = 'mic';
        } else if (selectedSource === 'file') {
            // Switch to pre-recorded file
            mic.disconnect(); // Disconnect microphone from master
output
            sound.disconnect(); // Ensure sound is connected
properly
            sound.connect(); // Connect sound to master output
            currentSource = 'file';
        }
    });
}

// Function to bind HTML playback buttons
function bindPlaybackControls() {
    document.getElementById('playButton').addEventListener('click',

```

```

() => {
    if (!sound.isPlaying()) sound.play();
});

document.getElementById('pauseButton').addEventListener('click',
() => {
    if (sound.isPlaying()) sound.pause();
});

document.getElementById('stopButton').addEventListener('click',
() => {
    sound.stop();
});

document.getElementById('skipStartButton').addEventListener('click',
() => {
    sound.jump(0);
});

document.getElementById('skipEndButton').addEventListener('click',
() => {
    if (sound.isLoaded()) sound.jump(sound.duration());
});

document.getElementById('loopButton').addEventListener('click',
(e) => {
    loopEnabled = !loopEnabled;
    sound.setLoop(loopEnabled);
    e.target.innerText = loopEnabled ? 'Loop: On' : 'Loop: Off';
});

document.getElementById('muteButton').addEventListener('click',
(e) => {
    muted = !muted;
    if (muted) {
        sound.setVolume(0);
        e.target.innerText = 'Mute: On';
    } else {
        const volume =
parseFloat(document.getElementById('volumeSlider').value);
        sound.setVolume(volume);
        e.target.innerText = 'Mute: Off';
    }
});

document.getElementById('volumeSlider').addEventListener('input',
(e) => {
    const linearValue = parseFloat(e.target.value); // Get the
slider value
    const smoothedValue = Math.pow(linearValue, 2); // Apply
quadratic scaling for smoother adjustment

```

```

        document.getElementById('volumeValue').innerText = `Master
Volume Level = ${smoothedValue.toFixed(2)}x`; // Update the display
        if (!muted) {
            sound.setVolume(smoothedValue); // Adjust the volume if
not muted
        }
    });

```

```

document.getElementById('recordButton').addEventListener('click',
(e) => {
    if (!isRecording) {
        recorder.record(recordingFile);
        e.target.innerText = 'Stop Recording';
    } else {
        recorder.stop();
        save(recordingFile, 'processed_audio.wav');
        e.target.innerText = 'Record';
    }
    isRecording = !isRecording;
});
}

```

```

// Filter controls
function bindFilterControls() {
    const filterToggleButton =
document.getElementById('filterToggleButton');
    const filterTypeSelector =
document.getElementById('filterTypeSelector');
    const freqSlider = document.getElementById('freqSlider');
    const freqValue = document.getElementById('freqValue');
    const resSlider = document.getElementById('resSlider');
    const resValue = document.getElementById('resValue');

    // Handle filter type change
    filterTypeSelector.addEventListener('change', (e) => {
        const filterType = e.target.value; // Get selected filter
type
        filter.setType(filterType); // Update the filter type
    });

    // Toggle filter on/off
    filterToggleButton.addEventListener('click', () => {
        filterEnabled = !filterEnabled; // Toggle filter state
        if (filterEnabled) {
            sound.disconnect(); // Disconnect sound from master
output
            sound.connect(filter); // Connect sound through filter
            filter.connect(); // Connect filter to master output
            processedFFT.setInput(filter); // Analyze the processed
signal
            filterToggleButton.innerText = 'Filter: On';
        } else {
            sound.disconnect(); // Disconnect filter

```

```

        sound.connect(); // Connect sound directly to master
output
        processedFFT.setInput(); // Reset to analyze unprocessed
signal
        filterToggleButton.innerText = 'Filter: Off';
    }
});

// Update filter frequency
freqSlider.addEventListener('input', () => {
    const freq = parseFloat(freqSlider.value);
    freqValue.innerText = freq; // Update the displayed value
    if (filterEnabled) filter.freq(freq);
});

// Update filter resonance
resSlider.addEventListener('input', () => {
    const res = parseFloat(resSlider.value);
    resValue.innerText = res; // Update the displayed value
    if (filterEnabled) filter.res(res);
});

// Update dry/wet mix
dryWetSlider.addEventListener('input', () => {
    const dryWet = parseFloat(dryWetSlider.value);
    dryWetValue.innerText = dryWet.toFixed(2); // Display the
dry/wet value
    if (filterEnabled) {
        filter.drywet(dryWet); // Set dry/wet mix for the filter
    }
});

// Update output level
outputLevelSlider.addEventListener('input', () => {
    const outputLevel = parseFloat(outputLevelSlider.value);
    outputLevelValue.innerText = outputLevel.toFixed(2); //
Display the output level
    if (filterEnabled) {
        filter.amp(outputLevel); // Adjust the output level of
the filter
    }
});
}

// function to bind waveshaper controls
function bindWaveshaperControls() {
    const waveshaperToggleButton =
document.getElementById('waveshaperToggleButton');
    const distortionAmountSlider =
document.getElementById('distortionAmountSlider');
    const distortionAmountValue =
document.getElementById('distortionAmountValue');
    const waveshaperOversampleButton =
document.getElementById('waveshaperOversampleButton');

```

```

    const waveshaperDryWetSlider =
document.getElementById('waveshaperDryWetSlider');
    const waveshaperDryWetValue =
document.getElementById('waveshaperDryWetValue');
    const waveshaperOutputSlider =
document.getElementById('waveshaperOutputSlider');
    const waveshaperOutputValue =
document.getElementById('waveshaperOutputValue');

    // Toggle Waveshaper
    waveshaperToggleButton.addEventListener('click', () => {
        waveshaperEnabled = !waveshaperEnabled;
        if (waveshaperEnabled) {
            sound.disconnect(); // Disconnect sound from master
output
            sound.connect(waveshaper); // Route sound through
Waveshaper
            processedFFT.setInput(waveshaper);
            waveshaperToggleButton.innerText = 'Waveshaper: On';
        } else {
            sound.disconnect(); // Disconnect Waveshaper
            sound.connect(); // Connect sound directly to master
output
            processedFFT.setInput();
            waveshaperToggleButton.innerText = 'Waveshaper: Off';
        }
    });

    // Adjust Distortion Amount
    distortionAmountSlider.addEventListener('input', () => {
        const amount = parseFloat(distortionAmountSlider.value);
        distortionAmountValue.innerText = amount.toFixed(2);
        waveshaper.set(amount, oversampleEnabled ? '2x' :
'none'); // Apply distortion and oversampling
    });

    // Toggle Oversampling
    waveshaperOversampleButton.addEventListener('click', () => {
        oversampleEnabled = !oversampleEnabled;
        const mode = oversampleEnabled ? '2x' : 'none';
        waveshaper.set(parseFloat(distortionAmountSlider.value),
mode); // Update oversampling
        waveshaperOversampleButton.innerText = `Oversample: $
{oversampleEnabled ? 'On' : 'Off'}`;
    });

    // Adjust Dry/Wet Mix
    waveshaperDryWetSlider.addEventListener('input', () => {
        const dryWet = parseFloat(waveshaperDryWetSlider.value);
        waveshaperDryWetValue.innerText = dryWet.toFixed(2);
        waveshaper.drywet(dryWet);
    });

    // Adjust Output Level

```

```

        waveshaperOutputSlider.addEventListener('input', () => {
            const outputLevel =
parseFloat(waveshaperOutputSlider.value);
            waveshaperOutputValue.innerText = outputLevel.toFixed(2);
            sound.setVolume(outputLevel); // Adjust final output level
        });
    }

// function to bind delay controls
function bindDelayControls() {
    const delayToggleButton =
document.getElementById('delayToggleButton');
    const delayTimeSlider =
document.getElementById('delayTimeSlider');
    const delayTimeValue =
document.getElementById('delayTimeValue');
    const feedbackSlider =
document.getElementById('feedbackSlider');
    const feedbackValue = document.getElementById('feedbackValue');
    const filterSlider = document.getElementById('filterSlider');
    const filterValue = document.getElementById('filterValue');

    // Toggle delay on/off
    delayToggleButton.addEventListener('click', () => {
        delayEnabled = !delayEnabled;
        if (delayEnabled) {
            sound.disconnect(); // Disconnect sound from master
output
            sound.connect(delay); //
            processedFFT.setInput(delay);
            delayToggleButton.innerText = 'Delay: On';
        } else {
            sound.disconnect(); // Disconnect delay
            sound.connect(); // Connect sound directly to master
output
            processedFFT.setInput();
            delayToggleButton.innerText = 'Delay: Off';
        }
    });

    // Adjust delay time
    delayTimeSlider.addEventListener('input', () => {
        const delayTime = parseFloat(delayTimeSlider.value);
        delayTimeValue.innerText = `${delayTime}s`;
        delay.delayTime(delayTime); // Update delay time
    });

    // Adjust feedback
    feedbackSlider.addEventListener('input', () => {
        const feedback = parseFloat(feedbackSlider.value);
        feedbackValue.innerText = `${(feedback * 100).toFixed(0)}%`;
        delay.feedback(feedback); // Update feedback
    });

```

```

// Adjust filter frequency
filterSlider.addEventListener('input', () => {
    const filterFreq = parseFloat(filterSlider.value);
    filterValue.innerText = `${filterFreq}Hz`;
    delay.filter(filterFreq); // Update filter frequency
});
}

// function to bind compressor controls
function bindCompressorControls() {
    const compressorToggleButton =
document.getElementById('compressorToggleButton');
    const thresholdSlider =
document.getElementById('thresholdSlider');
    const thresholdValue =
document.getElementById('thresholdValue');
    const ratioSlider = document.getElementById('ratioSlider');
    const ratioValue = document.getElementById('ratioValue');
    const attackSlider = document.getElementById('attackSlider');
    const attackValue = document.getElementById('attackValue');
    const releaseSlider = document.getElementById('releaseSlider');
    const releaseValue = document.getElementById('releaseValue');
    const kneeSlider = document.getElementById('kneeSlider');
    const kneeValue = document.getElementById('kneeValue');
    const compressorDryWetSlider =
document.getElementById('compressorDryWetSlider');
    const compressorDryWetValue =
document.getElementById('compressorDryWetValue');
    const compressorOutputLevelSlider =
document.getElementById('compressorOutputLevelSlider');
    const compressorOutputLevelValue =
document.getElementById('compressorOutputLevelValue');

    // Toggle Compressor
    compressorToggleButton.addEventListener('click', () => {
        compressorEnabled = !compressorEnabled;
        if (compressorEnabled) {
            sound.disconnect();
            sound.connect(compressor); // Route sound through
compressor
            compressor.connect(); // Connect compressor to master
output
            processedFFT.setInput(compressor);
            compressorToggleButton.innerText = 'Compressor: On';
        } else {
            sound.disconnect();
            sound.connect(); // Connect sound directly to master
output
            processedFFT.setInput();
            compressorToggleButton.innerText = 'Compressor: Off';
        }
    });

    // Adjust Threshold

```

```

thresholdSlider.addEventListener('input', () => {
    const threshold = parseFloat(thresholdSlider.value);
    thresholdValue.innerText = `${threshold} dB`;
    compressor.threshold(threshold);
});

// Adjust Ratio
ratioSlider.addEventListener('input', () => {
    const ratio = parseFloat(ratioSlider.value);
    ratioValue.innerText = ratio;
    compressor.ratio(ratio);
});

// Adjust Attack
attackSlider.addEventListener('input', () => {
    const attack = parseFloat(attackSlider.value) / 1000; //
Convert to seconds
    attackValue.innerText = `${attackSlider.value} ms`;
    compressor.attack(attack);
});

// Adjust Release
releaseSlider.addEventListener('input', () => {
    const release = parseFloat(releaseSlider.value) / 1000; //
Convert to seconds
    releaseValue.innerText = `${releaseSlider.value} ms`;
    compressor.release(release);
});

// Adjust Knee
kneeSlider.addEventListener('input', () => {
    const knee = parseFloat(kneeSlider.value);
    kneeValue.innerText = `${knee} dB`;
    compressor.knee(knee);
});

// Adjust Compressor Dry/Wet Mix
compressorDryWetSlider.addEventListener('input', () => {
    const dryWet = parseFloat(compressorDryWetSlider.value);
    compressorDryWetValue.innerText = dryWet.toFixed(2);

    if (compressorEnabled) {
        // Adjust dry/wet mix for compressor
        compressor.drywet(dryWet);
    }
});

// Adjust Compressor Output Level
compressorOutputLevelSlider.addEventListener('input', () => {
    const outputLevel =
parseFloat(compressorOutputLevelSlider.value);
    compressorOutputLevelValue.innerText =
outputLevel.toFixed(2);

```



```

        if (compressorEnabled) {
            compressor.amp(outputLevel); // Amplify the compressed
signal
        } else {
            sound.setVolume(outputLevel); // Amplify the original
(dry) signal
        }
    });
}

```

```

// function to bind reverb controls
function bindReverbControls() {
    const reverbToggleButton =
document.getElementById('reverbToggleButton');
    const reverbDurationSlider =
document.getElementById('reverbDurationSlider');
    const reverbDurationValue =
document.getElementById('reverbDurationValue');
    const reverbDecaySlider =
document.getElementById('reverbDecaySlider');
    const reverbDecayValue =
document.getElementById('reverbDecayValue');
    const reverbDryWetSlider =
document.getElementById('reverbDryWetSlider');
    const reverbDryWetValue =
document.getElementById('reverbDryWetValue');
    const reverbOutputSlider =
document.getElementById('reverbOutputSlider');
    const reverbOutputValue =
document.getElementById('reverbOutputValue');
    const reverbReverseButton =
document.getElementById('reverbReverseButton');
}

```

```

// Toggle Reverb
reverbToggleButton.addEventListener('click', () => {
    reverbEnabled = !reverbEnabled; // Toggle state
    console.log(`Reverb Enabled: ${reverbEnabled}`); //
Debugging
}

```

```

    if (reverbEnabled) {
        // Enable reverb
        sound.disconnect(); // Disconnect from master output
        sound.connect(reverb); // Connect sound through reverb
        processedFFT.setInput(reverb);
        reverb.connect(); // Connect reverb to master output

        // Set reverb duration and decay for a pronounced effect
        reverb.set(2, 0.5); // Maximum duration (10 seconds) and
decay (1.0)
        reverbToggleButton.innerText = 'Reverb: On'; // Update
button text
    } else {
        // Disable reverb
        sound.disconnect(); // Disconnect from reverb
    }
}

```

```

        sound.connect(); // Connect sound directly to master
output
        processedFFT.setInput();
        reverbToggleButton.innerText = 'Reverb: Off'; // Update
button text
    }
    });

    // Adjust Reverb Duration
    reverbDurationSlider.addEventListener('input', () => {
        const duration = parseFloat(reverbDurationSlider.value);
        reverbDurationValue.innerText = duration.toFixed(1);
        if (reverbEnabled) {
            reverb.set(duration,
parseFloat(reverbDecaySlider.value)); // Update duration and decay
        }
    });

    // Adjust Reverb Decay Rate
    reverbDecaySlider.addEventListener('input', () => {
        const decay = parseFloat(reverbDecaySlider.value);
        reverbDecayValue.innerText = decay.toFixed(2);
        if (reverbEnabled) {
            reverb.set(parseFloat(reverbDurationSlider.value),
decay); // Update duration and decay
        }
    });

    // Adjust Reverb Dry/Wet Mix
    reverbDryWetSlider.addEventListener('input', () => {
        const dryWet = parseFloat(reverbDryWetSlider.value);
        reverbDryWetValue.innerText = dryWet.toFixed(2);
        if (reverbEnabled) {
            reverb.drywet(dryWet); // Blend dry and wet signals
        }
    });

    // Adjust Reverb Output Level
    reverbOutputSlider.addEventListener('input', () => {
        const outputLevel = parseFloat(reverbOutputSlider.value);
        reverbOutputValue.innerText = outputLevel.toFixed(2);
        sound.setVolume(outputLevel); // Adjust output volume
    });

    // Toggle Reverse
    reverbReverseButton.addEventListener('click', () => {
        reverbReverseEnabled = !reverbReverseEnabled; // Toggle
state
        console.log(`Reverb Reverse Enabled: ${reverbReverseEnabled}
`); // Debugging

        if (reverbReverseEnabled) {
            if (reverbEnabled) {
                // Enable reverse reverb

```

```

        sound.disconnect(); // Disconnect from master output
        sound.connect(reverb); // Connect sound through
reverb
        reverb.connect(); // Connect reverb to master output

        // Set reverb duration and decay to maximum for a
pronounced effect
        reverb.set(4, 1); // Maximum duration (10 seconds)
and decay (1.0)
    }
    reverbReverseButton.innerText = 'Reverse: On'; // Update
button text
    } else {
        if (reverbEnabled) {
            // Ensure reverb remains connected
            sound.disconnect(); // Disconnect from master output
            sound.connect(reverb); // Reconnect reverb
            reverb.connect(); // Connect reverb to master output
        } else {
            // If reverb is not enabled, connect sound directly
to master output
            sound.disconnect();
            sound.connect();
        }
        reverbReverseButton.innerText = 'Reverse: Off'; //
Update button text
    }
});
}

```

```

// -----
// Ex2.1 - index.html *Code by myself*
// -----
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Audio Captcha</title>
    <script src="libraries/p5.js"></script>
    <script src="libraries/p5.sound.js"></script>
    <script src="libraries/p5.sound.min.js"></script>
    <script src="libraries/p5.speech.js"></script>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin: 0;
            padding: 20px;
            background-color: #f4f4f9;
        }
        #captcha-container {
            display: inline-block;

```

```

        margin: 20px auto;
        text-align: center;
    }
    #instruction-box {
        font-size: 1rem;
        margin-bottom: 20px;
        font-weight: bold;
        color: #333;
    }
    button {
        font-size: 1rem;
        padding: 10px 20px;
        margin: 10px;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }
    /* Default button styles */
    button {
        transition: all 0.2s ease; /* Smooth transition for
click effect */
    }

    /* Click effect */
    .button-clicked {
        transform: scale(0.95); /* Slightly shrink the button */
        box-shadow: 0 0 8px rgba(0, 0, 0, 0.2); /* Add a subtle
shadow */
        opacity: 0.8; /* Reduce opacity */
    }
    input[type="text"] {
        font-size: 1rem;
        padding: 10px;
        margin: 10px;
        border: 1px solid #ccc;
        border-radius: 5px;
        width: 80%;
    }
    #result {
        margin-top: 10px;
        font-weight: bold;
    }
</style>
</head>
<body>
    <div id="instruction-box">Type in the sentence you hear.</div>
    <div id="instruction-box">Omit punctuation.</div>
    <div id="captcha-container">
        <button id="playCaptcha">▶ Play Captcha</button>
        <button id="refreshCaptcha">🔄 Refresh Captcha</button>
        <input type="text" id="userInput" placeholder="Type what you
hear..." />
        <button id="submitCaptcha">Submit</button>
        <div id="result">Status: Waiting for input...</div>

```

```

        </div>
        <script src="sketch.js"></script>
    </body>
</html>
// -----
// Ex2.1 - sketch.js
// -----
let captchaAudio; // Variable to store the audio playback
let captchaText = ""; // Captcha text manually set for each audio
file
let isPlaying = false; // State for the play button

// Array to hold audio file paths and their corresponding text
const audioFiles = [
    { path: 'sounds/parents.wav', text: 'I have lost my parents' },
    { path: 'sounds/suitcase.wav', text: 'Please I have lost my
suitcase' },
    { path: 'sounds/what_time.wav', text: 'What time is my plane' },
    { path: 'sounds/where.wav', text: 'Where are the restaurants and
shops' },
    { path: 'sounds/checkin.wav', text: 'Where is the check in
desk' },
];

function preload() {
    // Preload all audio files
    audioFiles.forEach(audio => {
        audio.audio = loadSound(audio.path);
    });
}

function setup() {
    noCanvas();

    // Initialize UI elements
    const playButton = document.getElementById('playCaptcha');
    const refreshButton = document.getElementById('refreshCaptcha');
    const submitButton = document.getElementById('submitCaptcha');
    const userInput = document.getElementById('userInput');
    const resultDisplay = document.getElementById('result');

    // Add click effect logic to buttons
    const buttons = [playButton, refreshButton, submitButton];
    buttons.forEach((button) => {
        button.addEventListener('click', () => {
            button.classList.add('button-clicked');
            setTimeout(() => button.classList.remove('button-
clicked'), 200); // Remove after 200ms
        });
    });

    // Set up the initial captcha
    refreshCaptcha();
}

```

```

// Play button logic
playButton.addEventListener('click', () => {
  if (!isPlaying) {
    applyAudioScramble(); // Apply audio effects before
    captchaAudio.play();
    playButton.innerText = "⏸"; // Change to pause during
    isPlaying = true;
  } else {
    captchaAudio.pause();
    playButton.innerText = "▶"; // Change back to play
    isPlaying = false;
  }

  captchaAudio.onended(() => {
    playButton.innerText = "▶"; // Reset to play button
    isPlaying = false;
  });
});

// Refresh button logic
refreshButton.addEventListener('click', () => {
  refreshCaptcha();
  resultDisplay.innerText = "New Captcha Generated!";
  resultDisplay.style.color = "black"; // Reset color
  userInput.value = ""; // Clear input box
});

// Submit button logic
submitButton.addEventListener('click', () => {
  const userAnswer = userInput.value.trim().toLowerCase();
  const expectedAnswer = captchaText.trim().toLowerCase();

  console.log("Expected Answer: ", expectedAnswer);
  console.log("User Answer: ", userAnswer);

  if (userAnswer === expectedAnswer) {
    resultDisplay.innerText = "✅ Success! Captcha
matched.";
    resultDisplay.style.color = "green";
  } else {
    resultDisplay.innerText = "❌ Incorrect! Please try
again.";
    resultDisplay.style.color = "red";
  }
});
}

// Refresh captcha with a new random audio file
function refreshCaptcha() {
  // Stop any current playback
  if (captchaAudio && captchaAudio.isPlaying()) {

```

```

        captchaAudio.stop();
    }

    // Select a random audio file
    const randomIndex = Math.floor(Math.random() *
audioFiles.length);
    const selectedAudio = audioFiles[randomIndex];

    captchaAudio = selectedAudio.audio; // Assign the selected audio
    captchaText = selectedAudio.text; // Set the corresponding text

    console.log("Captcha Audio Selected: ", selectedAudio.path);
    console.log("Captcha Text: ", captchaText);
}

// Apply scrambling effects to the audio
function applyAudioScramble() {
    if (!captchaAudio || !captchaAudio.isLoaded()) {
        console.error("Captcha audio is not loaded or undefined.");
        return;
    }

    // Initialize speech synthesis
    let speech = new p5.Speech(); // p5.Speech instance
    speech.setLang('en-US'); // Set language
    speech.setVolume(1); // Set volume to max

    // Apply random voice from available voices
    const voices = speech.listVoices();
    if (voices && voices.length > 0) {
        const randomVoice = voices[Math.floor(Math.random() *
voices.length)];
        speech.setVoice(randomVoice.name);
    }

    // Apply random pitch
    const randomPitch = random(0.5, 1.5); // Adjust pitch range as
needed
    speech.setPitch(randomPitch);

    // Apply random rate
    const randomRate = random(0.8, 1.2); // Adjust rate range as
needed
    speech.setRate(randomRate);

    // Generate text for speech synthesis
    const scrambledText = captchaText.split('').join(' '); //
Introduce spacing between letters

    // Set a softer volume for speech synthesis
    speech.setVolume(0.2); // Set volume to 20% for softer output

    // Speak the scrambled text with effects
    speech.speak(scrambledText);

```

```

}
// -----
// Ex2.2_2.3 - index.html *Code by myself*
// -----
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Audio Visualization</title>
  <script src="libraries/p5.js"></script>
  <script src="libraries/p5.sound.min.js"></script>
  <script src="libraries/meyda.min.js"></script>
  <script src="libraries/p5.speech.js"></script>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin: 0;
      padding: 0;
      background-color: #121212;
      color: white;
    }
    canvas {
      display: block;
      margin: 20px auto;
    }
    select, button {
      font-size: 1rem;
      padding: 10px 20px;
      margin: 10px;
      border: none;
      border-radius: 5px;
      background-color: #1e90ff;
      color: white;
      cursor: pointer;
      transition: background-color 0.3s ease;
    }
    select:hover, button:hover {
      background-color: #63b3ed;
    }
    select:focus, button:focus {
      outline: none;
    }
    #rmsButton {
      background-color: green; /* Default color for RMS off */
    }
    #rmsButton.active {
      background-color: orange; /* Color when RMS is toggled
on */
    }
    #spectralRolloffButton {
      background-color: green; /* Default color for RMS off */

```



```

    }
    #spectralRolloffButton.active {
        background-color: orange; /* Color when RMS is toggled
on */
    }
    #zcrButton {
        background-color: green; /* Default color for RMS off */
    }
    #zcrButton.active {
        background-color: orange; /* Color when RMS is toggled
on */
    }
    #voiceButton {
        background-color: grey; /* Default for Disabled */
    }
</style>
</head>
<body>
    <h1>Audio Visualization with Meyda</h1>
    <select id="audioDropdown">
        <option value="">Select an audio</option>
        <option value="Ex2_sound1.wav">Ex2_sound1</option>
        <option value="Ex2_sound2.wav">Ex2_sound2</option>
        <option value="Ex2_sound3.wav">Ex2_sound3</option>
        <option value="Kalte_Ohren_(Remix).mp3">Kalte_Ohren</
option>
    </select>
    <div id="controls">
        <button id="playButton">Play</button>
        <button id="pauseButton">Pause</button>
        <button id="stopButton">Stop</button>
        <button id="voiceButton">Voice Control: Disabled</button>
    </div>
    <div id="meydaControls">
        <button id="rmsButton">RMS Off</button>
        <button id="spectralRolloffButton">Spectral Rolloff Off</
button>
        <button id="zcrButton">ZCR Off</button>
    </div>
    <script src="sketch.js"></script>
</body>
</html>
// -----
// Ex2.2_2.3 - sketch.js *Code by myself*
// -----
let audioElement;
let audioContext;
let source;
let analyzer;
let isPlaying = false;
let voiceRec;
let isVoiceEnabled = false;
const selectedFeatures = [];
const circles = [];

```

```

const numCircles = 20;
let currentBackgroundColor = [0]; // Track the background color
separately
const shapesArray = []; // New array to store shape types

function setup() {
  createCanvas(800, 400);

  // Create the voice recognition object
  voiceRec = new p5.SpeechRec('en-US', gotSpeech);
  voiceRec.continuous = true;
  voiceRec.interimResults = true;

  // Request microphone access
  const mic = new p5.AudioIn();
  mic.start(() => {
    console.log("Microphone initialized successfully");
  }, (err) => {
    console.error("Microphone initialization failed:", err);
  });

  const dropdown = document.getElementById('audioDropdown');
  const playButton = document.getElementById('playButton');
  const pauseButton = document.getElementById('pauseButton');
  const stopButton = document.getElementById('stopButton');
  const rmsButton = document.getElementById('rmsButton');
  const spectralRolloffButton =
document.getElementById('spectralRolloffButton');
  const zcrButton = document.getElementById('zcrButton');
  const voiceButton = document.getElementById('voiceButton');

  dropdown.addEventListener('change', handleDropdownSelection);
  playButton.addEventListener('click', playAudio);
  pauseButton.addEventListener('click', pauseAudio);
  stopButton.addEventListener('click', stopAudio);
  rmsButton.addEventListener('click', () => toggleFeature('rms',
rmsButton));
  spectralRolloffButton.addEventListener('click', () =>
toggleFeature('spectralRolloff', spectralRolloffButton));
  zcrButton.addEventListener('click', () => toggleFeature('zcr',
zcrButton));
  voiceButton.addEventListener('click', toggleVoiceControl);

  // Initialize circle array
  for (let i = 0; i < numCircles; i++) {
    circles.push({
      x: map(i, 0, numCircles - 1, 50, width - 50),
      y: height / 2,
      size: 10,
      color: color(0, 255, 0),
      shape: 'circle' // Default shape
    });
  }
}

```

```

    audioContext = new (window.AudioContext ||
window.webkitAudioContext)();
}

// Function to toggle voice control
function toggleVoiceControl() {
    const voiceButton = document.getElementById('voiceButton');
    isVoiceEnabled = !isVoiceEnabled;

    if (isVoiceEnabled) {
        voiceRec.start();
        voiceButton.style.backgroundColor = 'orange';
        voiceButton.innerText = 'Voice Control: Enabled';
        console.log('Voice control enabled. Waiting for
commands...');
    } else {
        voiceRec.stop();
        voiceButton.style.backgroundColor = 'grey';
        voiceButton.innerText = 'Voice Control: Disabled';
        console.log('Voice control disabled. ');
    }
}

// Callback function when speech is detected
function gotSpeech() {
    if (voiceRec.resultValue) {
        const command = voiceRec.resultString.toLowerCase();
        console.log(`Voice Command: ${command}`);

        // Handle background color changes
        const colors = ["black", "white", "red", "blue", "green"];
        if (colors.includes(command)) {
            changeBackgroundColor(command);
            return;
        }

        // Handle shape changes
        const shapes = ["square", "triangle", "circle", "pentagon"];
        if (shapes.includes(command)) {
            changeShape(command);
            return;
        }

        console.log(`Unrecognized command: ${command}`);
    }
}

// Change the background color based on voice command
function changeBackgroundColor(colorName) {
    const colors = {
        black: color(0),
        white: color(255),
        red: color(255, 0, 0),
        blue: color(0, 0, 255),
    };

```

```

        green: color(0, 255, 0),
    };

    if (colors[colorName]) {
        currentBackgroundColor = colors[colorName]; // Update the
tracked background color
        console.log(`Background changed to ${colorName}`);
    }
}

function changeShape(shapeName) {
    console.log(`Shape changed to ${shapeName}`);
    for (let circle of circles) {
        circle.shape = shapeName; // Update the shape for each
circle
    }
}

function handleDropdownSelection(event) {
    const selectedAudio = event.target.value;
    if (selectedAudio) {
        const audioPath = `sounds/${selectedAudio}`;
        resetAudio();

        audioElement = new Audio(audioPath);
        source =
audioContext.createMediaElementSource(audioElement);

        updateAnalyzer();

        source.connect(audioContext.destination);

        audioElement.onended = resetAudio;
        console.log(`File loaded: ${selectedAudio}`);
    }
}

function playAudio() {
    if (audioElement && !isPlaying) {
        audioElement.play();
        if (analyzer) analyzer.start();
        isPlaying = true;
        loop();
    }
}

function pauseAudio() {
    if (audioElement && isPlaying) {
        audioElement.pause();
        if (analyzer) analyzer.stop();
        isPlaying = false;
        noLoop();
    }
}

```

```

function stopAudio() {
  if (audioElement) {
    audioElement.pause();
    audioElement.currentTime = 0;
  }
  if (analyzer) {
    analyzer.stop();
  }
  isPlaying = false;
  noLoop();
  resetCircles();
}

function resetAudio() {
  if (audioElement) {
    audioElement.pause();
    audioElement.currentTime = 0;
    isPlaying = false;
  }
  if (analyzer) {
    analyzer.stop();
  }
  noLoop();
  resetCircles();
}

function resetCircles() {
  for (let circle of circles) {
    circle.size = 10;
    circle.color = color(0, 255, 0);
  }
}

function toggleFeature(feature, buttonElement) {
  if (selectedFeatures.includes(feature)) {
    selectedFeatures.splice(selectedFeatures.indexOf(feature),
1);
    buttonElement.style.backgroundColor = "green";
    buttonElement.innerText = `${feature.charAt(0).toUpperCase()
+ feature.slice(1)} Off`;
    resetCircles();
  } else {
    selectedFeatures.push(feature);
    buttonElement.style.backgroundColor = "orange";
    buttonElement.innerText = `${feature.charAt(0).toUpperCase()
+ feature.slice(1)} On`;
  }
  updateAnalyzer();
}

function updateAnalyzer() {
  if (analyzer) analyzer.stop();
  if (selectedFeatures.length > 0) {

```

```

        analyzer = Meyda.createMeydaAnalyzer({
            audioContext: audioContext,
            source: source,
            bufferSize: 512,
            featureExtractors: selectedFeatures,
            callback: handleFeatures,
        });
        analyzer.start();
    }
}

function handleFeatures(features) {
    console.log("Features received:", features);
    if (features.rms || features.spectralRolloff || features.zcr) {
        updateCircles(features.rms, features.spectralRolloff,
features.zcr);
    }
}

function updateCircles(rms, spectralRolloff, zcr) {
    const maxSize = 200;
    const adjustedRMS = rms ? rms ** 0.3 : 0;

    // Log the ZCR value to the console
    console.log(`ZCR Value: ${zcr}`);

    for (let i = 0; i < circles.length; i++) {
        const energy = adjustedRMS;

        circles[i].size = map(energy, 0, 1, 10, maxSize);
        const red = map(energy, 0, 1, 0, 255);
        const green = map(energy, 0, 0.5, 255, 0);
        circles[i].color = color(red, green, 0);

        const borderThickness = spectralRolloff
            ? map(spectralRolloff, 0, 15000, 1, 50)
            : 1;
        circles[i].borderThickness = borderThickness;

        // New code to adjust bouncing speed based on ZCR
        if (zcr) {
            // Map ZCR values from 20 to 130 to a bounce speed range
            // (e.g., 1 to 20) with increased sensitivity
            const bounceSpeed = map(zcr, 20, 130, 0, 15); // Adjust
the range as needed
            const sensitivityFactor = 2; // Increase sensitivity
            const adjustedBounceSpeed = bounceSpeed **
sensitivityFactor; // Apply power for sensitivity

            // Initialize direction if not set
            if (typeof circles[i].direction === 'undefined') {
                circles[i].direction = (i % 2 === 0) ? 1 : -1; //
Alternate direction based on index
            }
        }
    }
}

```

```

        // Update position
        circles[i].y += circles[i].direction *
adjustedBounceSpeed * 0.1; // Bouncing effect

        // Reverse direction if touching the edges
        if (circles[i].y >= height) {
            circles[i].y = height; // Keep at the bottom edge
            circles[i].direction = -1; // Reverse direction to
up
        } else if (circles[i].y <= 0) {
            circles[i].y = 0; // Keep at the top edge
            circles[i].direction = 1; // Reverse direction to
down
        }
    }
}

function draw() {
    background(currentBackgroundColor); // Use the tracked
background color

    for (const circle of circles) {
        fill(circle.color);
        noStroke();

        // Draw based on the shape type
        if (circle.shape === 'circle') {
            ellipse(circle.x, circle.y, circle.size);
        } else if (circle.shape === 'square') {
            rect(circle.x - circle.size / 2, circle.y -
circle.size / 2, circle.size, circle.size);
        } else if (circle.shape === 'triangle') {
            triangle(circle.x, circle.y - circle.size / 2,
circle.size / 2,
circle.x + circle.size / 2, circle.y +
circle.size / 2);
        } else if (circle.shape === 'pentagon') {
            beginShape();
            for (let j = 0; j < 5; j++) {
                const angle = TWO_PI / 5 * j;
                const x = circle.x + cos(angle) * (circle.size / 2);
                const y = circle.y + sin(angle) * (circle.size / 2);
                vertex(x, y);
            }
            endShape(CLOSE);
        }

        // Draw the border based on the shape type
        stroke(255);
        strokeWeight(circle.borderThickness || 1);
        noFill();
    }
}

```

```

        // Draw border for the specific shape
        if (circle.shape === 'circle') {
            ellipse(circle.x, circle.y, circle.size); // Border for
circle
        } else if (circle.shape === 'square') {
            rect(circle.x - circle.size / 2, circle.y -
circle.size / 2, circle.size, circle.size); // Border for square
        } else if (circle.shape === 'triangle') {
            beginShape();
            for (let j = 0; j < 3; j++) {
                const angle = TWO_PI / 3 * j - HALF_PI; // Adjust
for triangle orientation
                const x = circle.x + cos(angle) * (circle.size / 2);
                const y = circle.y + sin(angle) * (circle.size / 2);
                vertex(x, y);
            }
            endShape(CLOSE); // Border for triangle
        } else if (circle.shape === 'pentagon') {
            beginShape();
            for (let j = 0; j < 5; j++) {
                const angle = TWO_PI / 5 * j;
                const x = circle.x + cos(angle) * (circle.size / 2);
                const y = circle.y + sin(angle) * (circle.size / 2);
                vertex(x, y);
            }
            endShape(CLOSE); // Border for pentagon
        }
    }
}
// -----
// Ex3.1_audio_analysis - ipynb notebook *Code by myself*
// -----

```

```

import numpy as np
import scipy
import librosa
import matplotlib.pyplot as plt
import sounddevice as sd

```

Identify which soundfile contains ultrasonic frequencies.

```

# List of audio files to analyze
audio_files = ['Ex3_sound1.wav', 'Ex3_sound2.wav', 'Ex3_sound3.wav',
'Ex3_sound4.wav']

```

```

# Function to analyze audio files
def analyze_audio_files(files):
    for file in files:
        # Load the audio file
        y, sr = librosa.load(file, sr=None)

        # Perform a Fourier Transform to get frequency content
        D = np.abs(librosa.stft(y))

```



```

    freqs = librosa.fft_frequencies(sr=sr)

    # Identify ultrasonic frequencies (above 20 kHz)
    ultrasonic_indices = np.where(freqs > 20000)[0]
    ultrasonic_content = D[ultrasonic_indices, :].sum(axis=0)

    # Check if there is significant ultrasonic content
    if np.any(ultrasonic_content > threshold): # Set a
threshold for detection
        print(f"{file} contains ultrasonic frequencies.")
        # Optionally, you can store the detected file for
further processing
        return file, y, sr

# Set a threshold for ultrasonic detection
threshold = 0.1 # Adjust this value based on your analysis
suspicious_file, audio_data, sample_rate =
analyze_audio_files(audio_files)
Ex3_sound1.wav contains ultrasonic frequencies.
Decipher the four-number code from Ex3_sound1.wav

import numpy as np
import librosa
import sounddevice as sd

# Load the suspicious sound file
file_path = 'Ex3_sound1.wav' # Replace with your actual file path
audio_data, sample_rate = librosa.load(file_path, sr=None)

# Perform a Fourier Transform to get frequency content
D = np.abs(librosa.stft(audio_data))
freqs = librosa.fft_frequencies(sr=sample_rate)

# Identify the peaks in the frequency spectrum
magnitude = np.mean(D, axis=1) # Average over time
threshold = 0.1 * np.max(magnitude) # Set a threshold for peak
detection
peaks_indices = np.where(magnitude > threshold)[0]
detected_frequencies = freqs[peaks_indices]

# Define a mapping of frequencies to numbers
frequency_to_number = {
    261.63: 0, # C
    293.66: 1, # D
    329.63: 2, # E
    349.23: 3, # F
    392.00: 4, # G
    440.00: 5, # A
    493.88: 6, # B
    523.25: 7, # C
    587.33: 8, # D
    659.25: 9 # E
}

```

```

# Decipher the secret code from detected frequencies
secret_code = []
for freq in detected_frequencies:
    # Find the closest frequency in the mapping
    closest_freq = min(frequency_to_number.keys(), key=lambda x:
abs(x - freq))
    secret_code.append(frequency_to_number[closest_freq])

# Print the deciphered secret code
print("Deciphered Secret Code:", secret_code)

# Function to generate and play the sound for the secret code
def generate_sine_wave(frequency, duration, sample_rate):
    t = np.linspace(0, duration, int(sample_rate * duration),
endpoint=False)
    wave = 0.5 * np.sin(2 * np.pi * frequency * t) # Amplitude of
0.5
    return wave
A
# Play the deciphered secret code
duration = 0.1 # Duration of each tone in seconds
for number in secret_code:
    frequency = list(frequency_to_number.keys())[number] # Get the
frequency for the number
    wave = generate_sine_wave(frequency, duration, sample_rate)
    sd.play(wave, samplerate=sample_rate)
    sd.wait() # Wait until the sound has finished playing
Deciphered Secret Code: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
2, 7]
Answer: Deciphered four-number secret code: 1127

```

To confirm the remaining audio files don't contain secret codes, I test them as well, here are the results:

```

import numpy as np
import librosa
import threading

audio_files = ['Ex3_sound2.wav', 'Ex3_sound3.wav', 'Ex3_sound4.wav']

# Define a function to process each audio file
def process_audio_file(file_path):
    # Load the sound file
    audio_data, sample_rate = librosa.load(file_path, sr=None)

    # Perform a Fourier Transform to get frequency content
    D = np.abs(librosa.stft(audio_data))
    freqs = librosa.fft_frequencies(sr=sample_rate)

    # Identify the peaks in the frequency spectrum
    magnitude = np.mean(D, axis=1) # Average over time
    threshold = 0.1 * np.max(magnitude) # Set a threshold for peak
detection
    peaks_indices = np.where(magnitude > threshold)[0]

```

[illegible]

```
9, 9, 9, 9, 9, 9, 9, 9, 9, 9]
```

Therefore, Ex3_sound1.wav contains the secret code and it is 1127.

```
// -----
```

```
Ex3.2_embedding_hidden_messages *Code by myself*
```

```
// -----
```

```
import wave
import numpy as np
import random
Embedding Algorithm
```

```
import wave
import numpy as np
```

```
def embed_message(audio_file, message, output_file):
    # Read the audio file
    with wave.open(audio_file, 'rb') as wav:
        params = wav.getparams()
        frames = wav.readframes(params.nframes)

    # Convert frames to numpy array and create a writable copy
    audio_data = np.frombuffer(frames, dtype=np.int16).copy() #
    Use .copy() to make it writable

    # Convert message to binary
    message_bin = ''.join(format(ord(i), '08b') for i in message)
    message_length = len(message_bin)

    # Check if the message can fit in the audio data
    if message_length > len(audio_data):
        raise ValueError("Message is too long to fit in the audio
file.")

    # Embed the message into the least significant bits
    for i in range(message_length):
        audio_data[i] = (audio_data[i] & ~1) | int(message_bin[i])

    # Write the modified audio data to a new file
    with wave.open(output_file, 'wb') as wav_out:
        wav_out.setparams(params)
        wav_out.writeframes(audio_data.tobytes())

# Input secret message and generate embedded file
secret_message = "An eye for an eye makes the whole world blind"
embed_message('Ex3_sound5.wav', secret_message,
'Ex3_sound5_embedded.wav')
Extraction Algorithm
```

```
def extract_message(audio_file, message_length):
    # Read the audio file
    with wave.open(audio_file, 'rb') as wav:
        frames = wav.readframes(wav.getnframes())

    # Convert frames to numpy array
```

```

audio_data = np.frombuffer(frames, dtype=np.int16)

# Extract the message from the least significant bits
message_bin = ''
for i in range(message_length):
    message_bin += str(audio_data[i] & 1)

# Convert binary to string
message = ''.join(chr(int(message_bin[i:i+8], 2)) for i in
range(0, len(message_bin), 8))
return message

# Extraction of secret code
extracted_message = extract_message('Ex3_sound5_embedded.wav',
len(secret_message) * 8)
print("Extracted Message:", extracted_message)
Extracted Message: An eye for an eye makes the whole world blind
Testing with other message and file

import wave
import numpy as np

def embed_message(audio_file, message, output_file):
    # Read the audio file
    with wave.open(audio_file, 'rb') as wav:
        params = wav.getparams()
        frames = wav.readframes(params.nframes)

    # Convert frames to numpy array and create a writable copy
    audio_data = np.frombuffer(frames, dtype=np.int16).copy() #
Use .copy() to make it writable

    # Convert message to binary
    message_bin = ''.join(format(ord(i), '08b') for i in message)
    message_length = len(message_bin)

    # Check if the message can fit in the audio data
    if message_length > len(audio_data):
        raise ValueError("Message is too long to fit in the audio
file.")

    # Embed the message into the least significant bits
    for i in range(message_length):
        audio_data[i] = (audio_data[i] & ~1) | int(message_bin[i])

    # Write the modified audio data to a new file
    with wave.open(output_file, 'wb') as wav_out:
        wav_out.setparams(params)
        wav_out.writeframes(audio_data.tobytes())

# Input secret message and generate embedded file
secret_message = "Someday, everything will make perfect sense. So
for now, laugh at the confusion, smile through the tears, and keep
reminding yourself that everything happens for a reason"

```

```

embed_message('Ex3_sound4.wav', secret_message,
'Ex3_sound4_embedded.wav')
def extract_message(audio_file, message_length):
    # Read the audio file
    with wave.open(audio_file, 'rb') as wav:
        frames = wav.readframes(wav.getnframes())

    # Convert frames to numpy array
    audio_data = np.frombuffer(frames, dtype=np.int16)

    # Extract the message from the least significant bits
    message_bin = ''
    for i in range(message_length):
        message_bin += str(audio_data[i] & 1)

    # Convert binary to string
    message = ''.join(chr(int(message_bin[i:i+8], 2)) for i in
range(0, len(message_bin), 8))
    return message

# Extraction of secret code
extracted_message = extract_message('Ex3_sound4_embedded.wav',
len(secret_message) * 8)
print("Extracted Message:", extracted_message)
Extracted Message: Someday, everything will make perfect sense. So
for now, laugh at the confusion, smile through the tears, and keep
reminding yourself that everything happens for a reason
Hope this is correct.
// -----
Ex4_ASR_system *Code by myself*
// -----

```

Ex4.1: Automatic Speech Recognition (ASR) System

```

from deepspeech import Model
import librosa as lr
import numpy as np
import string
import unicodedata
import csv
from scipy.signal import butter, lfilter
from prettytable import PrettyTable
import threading
import time
from termcolor import colored

# Step 1: Initialize DeepSpeech models for English, Italian, and
Spanish
def initialize_models():
    models = {}

    # English model
    scorer_en = "Models/EN/deepspeech-0.9.3-models.scorer"
    model_path_en = "Models/EN/deepspeech-0.9.3-models.pbmm"

```

```

model_en = Model(model_path_en)
model_en.enableExternalScorer(scorer_en)
models["English"] = model_en

# Italian model
scorer_it = "Models/IT/kenlm_it.scorer"
model_path_it = "Models/IT/output_graph_it.pbmm"
model_it = Model(model_path_it)
model_it.enableExternalScorer(scorer_it)
models["Italian"] = model_it

# Spanish model
scorer_es = "Models/ES/kenlm_es.scorer"
model_path_es = "Models/ES/output_graph_es.pbmm"
model_es = Model(model_path_es)
model_es.enableExternalScorer(scorer_es)
models["Spanish"] = model_es

return models

# Step 2: Preprocess audio using ZCR to detect and reduce noise
def preprocess_audio_zcr(audio, sample_rate, threshold=0.1):
    zcr = lr.feature.zero_crossing_rate(audio, frame_length=2048,
hop_length=512)
    zcr_mean = np.mean(zcr)

    if zcr_mean < threshold:
        audio = audio * 1.5 # Apply gain for low ZCR
    else:
        audio = audio * 0.8 # Reduce gain for high ZCR

    return np.clip(audio, -1.0, 1.0) # Clip audio to valid range

# Step 3: Recognize speech from a single audio file
def recognize_audio(file_path, model, desired_sample_rate=16000):
    try:
        # Load the audio file using librosa
        audio, sample_rate = lr.load(file_path,
sr=desired_sample_rate)

        # Preprocess audio using ZCR
        audio = preprocess_audio_zcr(audio, sample_rate)

        # Scale audio to 16-bit range
        audio = (audio * 32767).astype(np.int16)

        # Perform speech recognition
        hypothesis = model.stt(audio)
        return hypothesis
    except Exception as e:
        print(f"Error processing {file_path}: {e}")
        return "Recognition Failed"

# Step 4: Preprocess text to handle accents, punctuation, and case

```

```

def preprocess_text(text):
    text = text.replace("-", " ").lower()
    text = text.translate(str.maketrans('', '', string.punctuation))
    text = ''.join(c for c in unicodedata.normalize('NFD', text) if
unicodedata.category(c) != 'Mn')
    return text

# Step 5: Compute Word Error Rate (WER)
def compute_wer(reference, hypothesis):
    ref_words = preprocess_text(reference).split()
    hyp_words = preprocess_text(hypothesis).split()

    dp = [[0 for _ in range(len(hyp_words) + 1)] for _ in
range(len(ref_words) + 1)]

    for i in range(len(ref_words) + 1):
        for j in range(len(hyp_words) + 1):
            if i == 0:
                dp[i][j] = j
            elif j == 0:
                dp[i][j] = i
            elif ref_words[i - 1] == hyp_words[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i
- 1][j - 1])

    wer = dp[len(ref_words)][len(hyp_words)] / len(ref_words) * 100
    return wer

# Spinner animation
def spinner_animation(stop_event):
    spinner = [" ", ".", ".. ", "..."]
    idx = 0
    while not stop_event.is_set():
        print(f"\rGenerating table, please wait{spinner[idx]}",
end="")
        idx = (idx + 1) % len(spinner)
        time.sleep(0.5)
    print("\rTable generation complete!          ")

# Step 6: Process all files from the CSV
def process_csv_transcriptions_with_spinner(csv_file_path, models):
    table = PrettyTable()
    table.field_names = ["Language", "File", "WER (%)"]
    table.align = "l" # Align text to the left
    table.align["WER (%)"] = "r" # Right align the WER column

    stop_event = threading.Event()
    spinner_thread = threading.Thread(target=spinner_animation,
args=(stop_event,))
    spinner_thread.start() # Start the spinner animation

    try:

```



```

        with open(csv_file_path, mode="r", encoding="utf-8-sig") as
csvfile:
            reader = csv.DictReader(csvfile)

            for row in reader:
                language = row["Language"]
                file_path = row["File"]
                reference_text = row["Transcriptions"]

                model = models.get(language)

                if model:
                    hypothesis = recognize_audio(file_path, model)
                    if hypothesis and hypothesis != "Recognition
Failed":
                        wer = compute_wer(reference_text,
hypothesis)
                        wer_color = colored(f"{wer:.2f}", "green" if
wer == 0.0 else "red")
                        table.add_row([language, file_path,
wer_color])
                    else:
                        table.add_row([language, file_path,
colored("Recognition Failed", "red")])
                else:
                    table.add_row([language, file_path, colored("No
Model", "red")])

            finally:
                stop_event.set() # Signal the spinner to stop
                spinner_thread.join() # Wait for the spinner thread to
finish

            print("\nRecognition Results:")
            print(table)

# Main execution
if __name__ == "__main__":
    models = initialize_models()
    csv_file_path = "transcriptions_03.csv"
    process_csv_transcriptions_with_spinner(csv_file_path, models)
    Table generation complete!

```

Recognition Results:

Language	File	WER (%)
English	Audio/EN/checkin.wav	0.00
English	Audio/EN/parents.wav	20.00
English	Audio/EN/suitcase.wav	0.00
English	Audio/EN/what_time.wav	20.00
English	Audio/EN/where.wav	0.00
English	Audio/EN/your_sentence1.wav	0.00
English	Audio/EN/your_sentence2.wav	0.00

Italian	Audio/IT/checkin_it.wav	25.00	
Italian	Audio/IT/parents_it.wav	20.00	
Italian	Audio/IT/suitcase_it.wav	0.00	
Italian	Audio/IT/what_time_it.wav	100.00	
Italian	Audio/IT/where_it.wav	42.86	
Spanish	Audio/ES/checkin_es.wav	25.00	
Spanish	Audio/ES/parents_es.wav	0.00	
Spanish	Audio/ES/suitcase_es.wav	0.00	
Spanish	Audio/ES/what_time_es.wav	83.33	
Spanish	Audio/ES/where_es.wav	42.86	
+-----+-----+-----+			